

GIT — Complete Command Reference & Daily Workflow

Setup · Everyday commands · Branching · Undo · Commit conventions · Real project workflow

1. ONE-TIME SETUP

Command	What it does
<code>git config --global user.name "Name"</code>	Set the author name recorded on every commit you make.
<code>git config --global user.email "you@mail.com"</code>	Set the author email recorded on every commit.
<code>git init</code>	Turn the current folder into a new Git repository.
<code>git clone <url></code>	Copy an existing remote repository to your machine.

2. THE DAILY WORKFLOW (USE THIS EVERY SESSION)

This is the loop you repeat every single day you work on a project — from your uploaded notes:

Command	What it does
<code>cd D:\Project\Portfolio</code>	Move into your project folder before doing anything else.
<code>git status</code>	Check what changed — modified, new, or deleted files.
<code>git add .</code>	Stage ALL changed files so they're ready to be committed.
<code>git commit -m "feat: your change"</code>	Save a snapshot of staged changes with a clear message.
<code>git push</code>	Upload your committed changes to GitHub / the remote repo.

Tip: run "git status" again after "git add ." and after "git commit" — it tells you exactly what will happen next.

3. ESSENTIAL EVERYDAY COMMANDS

Command	What it does
<code>git status</code>	Shows staged, unstaged, and untracked files right now.
<code>git add <file></code>	Stage one specific file instead of everything.
<code>git add .</code>	Stage every changed file in the current directory tree.
<code>git commit -m "msg"</code>	Record a snapshot of staged changes with a message.
<code>git commit -am "msg"</code>	Stage all tracked (already-known) files and commit in one step.
<code>git push origin main</code>	Send local commits on 'main' up to the remote named 'origin'.
<code>git pull</code>	Fetch and merge the latest changes from the remote branch.
<code>git log</code>	View the commit history for the current branch.
<code>git log --oneline</code>	Compact one-line-per-commit history — fast overview.
<code>git diff</code>	Show line-by-line changes not yet staged.
<code>git clone <url></code>	Download a full copy of a remote repo, including history.
<code>git rm <file></code>	Remove a file from the working directory and stage the deletion.
<code>git mv <old> <new></code>	Rename or move a file and stage the change in one step.

4. .GITIGNORE — KEEP JUNK OUT OF YOUR REPO

Create a file named .gitignore in your project root and list patterns for files Git should never track:

Command	What it does
<code>node_modules/</code>	Ignore installed npm dependencies — huge and regenerable.
<code>.env</code>	Never commit secrets, API keys, or local environment variables.
<code>*.log</code>	Ignore log files generated while the app runs.
<code>dist/ , build/</code>	Ignore compiled or bundled output folders.
<code>__pycache__/ , *.pyc</code>	Ignore Python bytecode cache files.

5. CHECKING WHERE YOU STAND

Command	What it does
<code>git remote -v</code>	Confirm which GitHub URL 'origin' points to before pushing.
<code>git branch -vv</code>	See every local branch and which remote branch it tracks.
<code>git log -1</code>	Show only the most recent commit — quick sanity check.

4. BRANCHING & MERGING

Command	What it does
<code>git branch</code>	List all local branches; current branch is starred.
<code>git branch <name></code>	Create a new branch without switching to it.
<code>git checkout -b <name></code>	Create a new branch and switch to it immediately.
<code>git switch <name></code>	Switch to an existing branch (modern replacement for checkout).
<code>git merge <branch></code>	Merge the named branch's changes into your current branch.
<code>git branch -d <name></code>	Delete a branch that's already been merged (safe delete).
<code>git branch -D <name></code>	Force-delete a branch even if it has unmerged work.
<code>git rebase <branch></code>	Replay your commits on top of another branch's history.

5. REMOTE & SYNC COMMANDS

Command	What it does
<code>git remote -v</code>	List remote repositories linked to this project and their URLs.
<code>git remote add origin <url></code>	Connect a local repo to a remote GitHub repository.
<code>git fetch</code>	Download new data from the remote without merging it yet.
<code>git pull origin main</code>	Fetch AND merge the latest 'main' branch from the remote.
<code>git push origin main</code>	Upload local commits on 'main' to the remote repository.
<code>git push -u origin <branch></code>	Push a new branch and set it to track the remote branch.

6. UNDO & FIX MISTAKES

Command	What it does
<code>git restore <file></code>	Discard unstaged changes in a file, reverting to last commit.
<code>git restore --staged <file></code>	Unstage a file but keep your edits in the working directory.
<code>git reset --soft HEAD~1</code>	Undo the last commit but keep changes staged.
<code>git reset --hard HEAD~1</code>	Undo the last commit and permanently discard its changes.
<code>git revert <commit></code>	Create a new commit that safely undoes a previous commit.
<code>git commit --amend -m "msg"</code>	Edit the message (or content) of the most recent commit.
<code>git stash</code>	Temporarily shelve uncommitted changes to switch tasks cleanly.
<code>git stash pop</code>	Reapply the most recently stashed changes.

7. INSPECTING HISTORY

Command	What it does
<code>git log --oneline --graph --all</code>	Visual branch/commit graph across all branches.
<code>git show <commit></code>	Show the full diff and metadata for one specific commit.
<code>git diff --staged</code>	Show changes that are staged and about to be committed.
<code>git blame <file></code>	Show which commit/author last changed each line of a file.

8. TAGS & RELEASES

Command	What it does
<code>git tag v1.0.0</code>	Mark the current commit as a release point, e.g. v1.0.0.
<code>git tag -a v1.0.0 -m "msg"</code>	Create an annotated tag with a message (recommended for releases).
<code>git push origin v1.0.0</code>	Push a single tag up to the remote repository.
<code>git push origin --tags</code>	Push every local tag to the remote at once.

9. WORKING WITH FORKS & MULTIPLE REMOTES

Command	What it does
<code>git remote add upstream <url></code>	Link the original repo you forked from as 'upstream'.
<code>git fetch upstream</code>	Download the latest changes from the original project.
<code>git merge upstream/main</code>	Bring upstream's latest 'main' into your local branch.

8. COMMIT MESSAGE PREFIXES (CONVENTIONAL COMMITS)

Using a prefix on every commit message makes your GitHub history instantly scannable. Format: <prefix>: <short description>.

Prefix	Use it for	Example
feat	A new feature or capability	feat: add PDF document upload
fix	A bug fix	fix: handle Gemini API errors
docs	Documentation only changes	docs: update project documentation
style	Formatting, no logic change	style: fix indentation in App.js
refactor	Code change, no new feature/fix	refactor: simplify chat handler logic
test	Adding or fixing tests	test: add unit tests for RAG retriever
chore	Maintenance / tooling / setup	chore: initialize NexaAI project
perf	A performance improvement	perf: speed up embedding lookup

9. REAL EXAMPLE — BUILDING THE "NEXAAI" PROJECT

A realistic sequence of commits as features are added one by one:

Command	What it does
chore: initialize NexaAI project	First commit — project scaffold created.
feat: add initial frontend and improve backend	Base UI and backend wired up.
feat: add chat interface	Chat UI component added.
feat: connect frontend to backend	Frontend now talks to the API.
fix: handle empty messages	Bug fix for a blank-message edge case.
feat: add PDF upload	Users can upload PDF documents.
feat: implement RAG retrieval	Retrieval-augmented generation added for PDFs.
docs: update README	Documentation brought up to date.

10. BAD HABITS VS. GOOD PRACTICE

Command	What it does
update / changes / final / final2 / working / new changes	✗ Vague, uninformative — avoid these entirely.
feat: add PDF document upload	✓ Clear, specific, tells reviewers exactly what changed.
fix: handle Gemini API errors	✓ Explains the bug fixed and where.

11. QUICK CHEAT SHEET — THE LOOP TO MEMORIZE

```
git status → see what changed
git add . → stage everything
git commit -m "feat: ..." → snapshot with a clear message
git push → upload to GitHub
```

Run this exact loop every time you sit down to work — check status, stage, commit with a meaningful prefixed message, then push.

12. COMMON ERRORS & QUICK FIXES

Command	What it does
"failed to push some refs"	Remote has newer commits — run "git pull" first, then push again.
"fatal: not a git repository"	You're outside the project folder, or forgot "git init".
Merge conflict markers <<<< ==== >>>>	Edit the file to keep the correct lines, then "git add" + "git commit".
Committed to the wrong branch	"git reset --soft HEAD~1", switch branch, then re-commit.
Forgot to add a file to last commit	"git add <file>" then "git commit --amend --no-edit".

13. USEFUL SHORTCUTS (ALIASES)

Add these once and reuse forever — saves typing on every commit:

Command	What it does
git config --global alias.st status	Now "git st" works as a shortcut for "git status".
git config --global alias.co checkout	Now "git co <branch>" works as a shortcut for checkout.

Command	What it does
<code>git config --global alias.cm "commit -m"</code>	Now "git cm \"msg\"" commits instantly.

* GIT - Everything You Need to Know

- ① `git add .`
- ② `git commit -m "feat: improve project section"`
- ③ `git push origin main`

Use daily once in a day:

- ① `cd D:\Project\Portfolio`
- ② `git status`
- ③ `git add .`
- ④ `git commit -m "feat: your change"`
- ⑤ `git push origin main`
- ⑥ `git status`
- ⑦ `git add .`
- ⑧ `git status`
- ⑨ `git commit -m "feat: describe what you added"`
- ⑩ `git push`

Common commit prefixes

Prefix	Meaning	Example
feat	New feature/ functionality	feat: add PDF upload
fix	Bug fix	fix: resolve chat API error
chore	Maintenance/ setup	chore: update dependencies
docs	Documentation	docs: update README
refactor	Improve code without changing behavior	refactor: simplify chat service
test	Add/update tests	test: add conversation API tests
style	Formatting/ style changes	style: format backend code

For your NexaAI project

- Imagine you add PDF uploading:
`git commit -m "feat: add PDF document upload"`
- Then you implement PDF text extraction:
`git commit -m "feat: add PDF text extraction"`
- Then RAG:
`git commit -m "feat: implement RAG document retrieval"`
- If you find a bug in Gemini responses:
`git commit -m "fix: handle Gemini API errors"`
- If you only update your README:
`git commit -m "docs: update project documentation"`

Why I'm recommending this

It makes your GitHub history easy to understand.

Instead of:

- update
- changes
- final
- final2
- working
- new changes



you get:

- ✓ chore: initialize NexaAI project
- ✓ feat: add initial frontend and improve backend
- ✓ feat: add chat interface
- ✓ feat: connect frontend to backend
- ✓ fix: handle empty messages
- ✓ feat: add PDF upload
- ✓ feat: implement RAG retrieval
- ✓ docs: update README

Anyone looking at your repository can quickly understand how the project evolved.



So whenever we make a commit together, I'll suggest the appropriate prefix (feat, fix, docs, etc.) based on what you actually changed. ✨